RETAIL DATA AGENT · RESILIENCE

When a query fails, who fixes it?

The AI writes SQL, and our gateway or BigQuery sometimes rejects it. If the problem is in the query, the AI gets the error and tries again. If the problem is BigQuery itself or our setup, retrying can't help, so the agent stops and tells the user.

BigQuery rejects a query
and sends back an error with a short reason code

Is it an infrastructure or setup problem?

NO → SEND IT TO THE AI

The AI reads the error, fixes the SQL and tries again

A typo in a column name
"Unrecognized name: revenue"

A SQL syntax mistake
a missing comma, a wrong function

A table that doesn't exist
a misspelled table name

The query would cost too much
it would scan more than our 1 GB cap, so the AI adds filters (normally caught by the gateway first, see below)

The query is too heavy, or the result too big
the AI aggregates or adds a LIMIT

Any error we haven't seen before
by default, the AI gets a chance

Usually ends in a normal answer. The user sees both the failed and the fixed SQL.

YES → DON'T SEND IT TO THE AI

The agent stops and shows a clear message

BigQuery is having problems
a server error or a temporary outage

Too many requests at once
rate limit reached

A table is temporarily unavailable
or the job was stopped

No permission
the account can't access the data

Billing or quota
billing is off, or the monthly free quota is used up

Blocked by a policy
an organization rule forbids it

The chat keeps running; the user can ask again. Step 4 will retry the temporary ones automatically.

Before BigQuery: the gateway's own checks

Every query passes our gateway first. If it breaks a rule, it never reaches BigQuery, costs nothing, and the reason goes to the AI like any query mistake.

REJECTED BY THE GATEWAY → SEND IT TO THE AI

Not a SELECT
INSERT, DELETE, CREATE and scripts are refused: "Only SELECT queries are allowed."

More than one statement
"Send one statement per query."

A table outside the 4 allowed ones
the error lists the tables it may use

A table function
such as ML.PREDICT, which reads data in other ways

SQL we can't parse
refused rather than run unchecked, so the AI simplifies it

Over the 1 GB limit
a free dry-run estimates the scan first: "This query would scan 2.1 GB, over the 1.0 GB limit per query..."

Example: a typo, fixed in one retry

1 **Manager asks:** "What is our revenue?" The AI writes SQL with a column that doesn't exist.

```
SELECT revenue FROM order_items
```

2 **BigQuery rejects it.** A query problem, so the error goes back to the AI.

```
Error: Unrecognized name: revenue at [1:8]
```

3 **The AI fixes the SQL** using the real column.

```
SELECT SUM(sale_price) AS revenue FROM order_items
```

4 **It works.** The manager gets the answer, plus both SQL statements, so it's clear what happened.

This used 3 AI calls: write, fix, answer.

Why retries can't run up the bill

5
AI calls per question, at most
A normal question uses 2. The 5th call can't run queries, so the AI answers from what it already has and says what is missing. The chat continues.

1 GB
scanned per query, at most
The gateway checks the estimate before running, and BigQuery enforces the same cap as a backstop. Either refusal goes to the AI so it can narrow the query.

0
retries on infrastructure errors
The AI never retries a problem it can't fix, such as an outage or a missing permission.

> **The rule in one sentence**
> We list only the problems the AI *can't* fix, and send everything else to it. SQL mistakes come in endless forms, while infrastructure problems are a short, known list, and the 5-call cap keeps the cost of a wrong guess small.

For engineers: the list is `_INFRASTRUCTURE_REASONS` in `src/retail_agent/db/bigquery.py`; errors without a reason code are decided by HTTP status (5xx, 401, 403 and 429 stop; others go to the AI). Gateway rejections are `QueryRejected` (a `QueryError`) from `query_safety/step1_guard.py` and `query_safety/gateway.py`. The 5-call cap is `CallBudget` in `src/retail_agent/agent.py`, set by `MAX_LLM_CALLS`.

Tests: `tests/test_bigquery_errors.py`, `tests/test_step1_guard.py` and `tests/test_turn.py`. Ticket: `tasks/03-recover-from-bad-sql.md`.